

Reviewer Pack — Free Entropy Lower Bound for Disjointness Communication Comp...

Entry b4aca3359472 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 17:04:06 UTC

Free Entropy Lower Bound for Disjointness Communication Complexity

Entry ID: b4aca3359472 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 17:04:06 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For a DISJOINTNESS matrix $M \in \{0,1\}^{\{n \times n\}}$, define $\tau(M)$ as the free entropy of the random variable $X = (M_{\{i,j\}} + 1)/2$ under the semicircular distribution. Then $\tau(M) \geq \Omega(n)$ for all M with support size $\geq n^{\{1.5\}}$.

Rationale (proposer's reasoning):

Free entropy quantifies non-commutative complexity via Voiculescu's entropy formula. DISJOINTNESS requires $\Omega(n)$ bits due to its combinatorial structure, and free entropy captures this via matrix-valued distributions over non-commutative algebras.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 14337f62263443b1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    M[i][j] = 1
                    M[j][i] = 1
        return M

    def free_entropy(M):
        X = [[(M[i][j] + 1) / 2 for j in range(len(M))] for i in range(len(M))]
        rho = spectral_radius(X)
        return -math.log(rho)

    def spectral_radius(matrix):
        n = len(matrix)
        v = [1.0] * n
        tol = 1e-6
        max_iter = 1000
        for _ in range(max_iter):
            w = [sum(matrix[i][j] * v[j] for j in range(n)) for i in range(n)]
            norm_w = sum(w[i]**2 for i in range(n))**0.5
            if abs(norm_w - 1) < tol:
                break
            v = [w[i] / norm_w for i in range(n)]
        return max(abs(v[i]) for i in range(n))

    n = 16
    M = generate_disjointness_matrix(n)
    tau_M = free_entropy(M)

    return {
        "metric_name": "free_entropy",
        "metric_value": tau_M,
```

```

        "instances_tested": 1,
        "conjecture_holds": tau_M >= 0.3 * n,
        "counterexample": "" if tau_M >= 0.3 * n else "tau(M) < 0.3n"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

n'}
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.239300320182068, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.239544873717278, 'instances_tested': 1, 'co
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2798415306829964, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2061111562331812, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2485277782386066, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2464538909911635, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2873156682669213, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.2887115871569925, 'instances_tested': 1, 'c
TRIAL: {'metric_name': 'free_entropy', 'metric_value': 1.1916827655139577, 'instances_tested': 1, 'c

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f90acabd.py", line 79, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f90acabd.py", line 79, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before completing validation; counterexamples shown may be unreliable due to missing seed data | next: Fix test to handle seed tracking and re-run with proper error handling

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107946
2	propose	ollama_remote	qwen3:8b	0	0	116953
3	propose	ollama_remote	qwen3:8b	0	0	41144
4	preregistration	ollama_remote	qwen3:8b	0	0	27715
5	novelty	ollama_remote	qwen3:8b	0	0	24031
6	novelty	ollama_remote	qwen3:8b	0	0	19231
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37145
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9356
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7664
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9857
11	judge	ollama_remote	qwen3:8b	0	0	22373

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 423414 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/b4aca3359472.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b4aca3359472.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b4aca3359472.tar.gz (if generated)